Python File Handling and Multithreading

Prepare notes on file handling in python with lots of example and there explanation
make sure you create note on each subtopic

Great 👍 Before we start — quick check:

Are you learning Python for **college exams**, **interviews**, or for **practical development (projects/automation)**?

Since you're from a programming background and preparing seriously, I'll structure this like **exam + practical oriented notes** — but I won't overload you. We'll build step-by-step.

📂 FILE HANDLING IN PYTHON – COMPLETE NOTES

1️⃣ What is File Handling?

🔹 Definition

File handling allows Python programs to **create, read, write, update, and delete files** stored on the system.

🔹 Why is it needed?

If a program ends, all variables are lost from memory (RAM).
Files help in **permanent storage**.

Example use cases:

- Storing user data
- Logs
- Reports
- Configuration files

2️⃣ Opening a File – `open()` Function

🔹 Syntax

```python
file_object = open("filename", "mode")
```

🔹 Example

```python
f = open("data.txt", "r")
```

Here:

- `"data.txt"` → file name
- `"r"` → mode (read mode)

🧠 Question for You:

What do you think will happen if the file does not exist and we use `"r"` mode?

(Think before scrolling.)

👉 It throws **FileNotFoundError**.

3️⃣ File Modes (VERY IMPORTANT)

Mode	Meaning
`"r"`	Read
`"w"`	Write (overwrites file)
`"a"`	Append
`"x"`	Create (error if exists)
`"b"`	Binary mode
`"t"`	Text mode (default)

🔹 Example 1: Write Mode (`w`)

```python
f = open("demo.txt", "w")
f.write("Hello Chetan")
f.close()
```

✔️ If file exists → content deleted

✔️ If file does not exist → created

🔹 Example 2: Append Mode (`a`)

```python
f = open("demo.txt", "a")
f.write("\nNew Line Added")
f.close()
```

✔️ Adds content at end

✔️ Does NOT delete old content

🔹 Example 3: Create Mode (`x`)

```python
f = open("newfile.txt", "x")
```

✔️ Creates new file

❌ Error if file already exists

4️⃣ Closing a File – `close()`

Always close file after use.

```python
f = open("demo.txt", "r")
print(f.read())
f.close()
```

Why?

- Frees system resources
- Saves changes properly

5️⃣ Using `with` Statement (Best Practice ⭐)

Instead of manually closing file:

```python
with open("demo.txt", "r") as f:
    data = f.read()
    print(data)
```

✔️ Automatically closes file
✔️ Cleaner and safer

🧠 Quick Check:

Why is `with` better than manually calling `close()`?

6️⃣ Reading from File

🔹 1. `read()`

Reads entire file.

```python
with open("demo.txt", "r") as f:
    print(f.read())
```

🔹 2. `read(n)`

Reads first `n` characters.

```python
with open("demo.txt", "r") as f:
    print(f.read(5))
```

🔹 3. `readline()`

Reads one line.

```python
with open("demo.txt", "r") as f:
    print(f.readline())
```

🔹 4. `readlines()`

Returns list of lines.

```python
with open("demo.txt", "r") as f:
    lines = f.readlines()
    print(lines)
```

Output:

```
['Hello\n', 'World\n']
```

7️⃣ Writing to File

🔹 `write()`

```python
with open("demo.txt", "w") as f:
    f.write("Line 1\n")
    f.write("Line 2\n")
```

🔹 `writelines()`

```python
lines = ["A\n", "B\n", "C\n"]
```

```python
with open("demo.txt", "w") as f:
    f.writelines(lines)
```

Note:

- Does NOT add newline automatically
- You must include `\n`

8️⃣ File Pointer Methods

🔹 `tell()`

Returns current cursor position.

```python
with open("demo.txt", "r") as f:
    print(f.tell())
```

🔹 `seek(position)`

Move cursor to position.

```python
with open("demo.txt", "r") as f:
    f.seek(5)
    print(f.read())
```

🧠 Concept:

File works like a cursor moving over text.

9️⃣ Checking if File Exists

Using `os` module:

```python
import os
```

```python
if os.path.exists("demo.txt"):
    print("File exists")
```

---

10 Deleting a File

```python
import os
os.remove("demo.txt")
```

---

11 Working with Binary Files

Used for:

- Images
- Videos
- PDFs

Example:

```python
with open("image.jpg", "rb") as f:
    data = f.read()
```

Binary write:

```python
with open("copy.jpg", "wb") as f:
    f.write(data)
```

---

12 Exception Handling with Files

```python
try:
    with open("demo.txt", "r") as f:
        print(f.read())
except FileNotFoundError:
    print("File not found!")
```

1️⃣3️⃣ Practical Example – Student Marks System

```python
with open("marks.txt", "w") as f:
    f.write("Chetan 85\n")
    f.write("Rahul 90\n")

with open("marks.txt", "r") as f:
    for line in f:
        name, marks = line.split()
        print(name, "scored", marks)
```

🧠 Mini Revision Summary

File Handling Steps:

1. Open file
2. Perform operation (read/write)
3. Close file (or use `with`)

Modes:

- `r` → Read
- `w` → Write
- `a` → Append
- `x` → Create
- `b` → Binary

Now let's make this interactive 👇

🔥 Quick Test (Answer one by one):

1. What happens if we open a file in `"w"` mode and file already exists?
2. What does `readlines()` return?
3. Why is `with` statement recommended?

Reply with answers. I'll correct and then we'll move to **advanced topics** like:

- CSV handling
- JSON handling
- Pickle
- Real interview-level file problems 🚀

prepare notes on Iterators, Generators, Closures and Decorators but in depth with exampel python full code for each and thre ambogious topic adn interview question and asnwer with them

Perfect 👍 This is an advanced Python topic set — very important for **interviews and deep Python understanding**.

Before we start:

You're from an engineering background — so tell me:
👉 Are you already comfortable with **functions, scope, and basic OOP in Python**?

(Just answer yes/no + your comfort level briefly.)

We'll cover:

1️⃣ Iterators
2️⃣ Generators
3️⃣ Closures
4️⃣ Decorators

- Ambiguous/confusing areas
- Interview Q&A
- Full working code
- Concept comparison

But remember — I won't just dump answers. I'll guide you step-by-step so you truly understand it.

🔹 PART 1: ITERATORS (Deep Dive)

1️⃣ What is an Iterator?

Definition:

An iterator is an object that:

- Implements `__iter__()`

- Implements `__next__()`
- Returns next value until `StopIteration`

🔥 Important Concept

Everything in:

```python
for x in something:
```

is internally converted into iterator protocol.

Example 1: Manual Iterator Using Built-in List

```python
nums = [1, 2, 3]

it = iter(nums)

print(next(it))  # 1
print(next(it))  # 2
print(next(it))  # 3
print(next(it))  # StopIteration
```

🧠 Question for You:

What do you think `iter(nums)` actually returns?

(Hint: Is it the list itself or a separate iterator object?)

Example 2: Custom Iterator Class (VERY IMPORTANT FOR INTERVIEW)

```python
class Counter:
    def __init__(self, max):
        self.max = max
        self.current = 0
```

```
    def __iter__(self):
        return self   # iterator returns itself

    def __next__(self):
        if self.current < self.max:
            self.current += 1
            return self.current
        else:
            raise StopIteration


counter = Counter(3)

for num in counter:
    print(num)
```

Output:

Code

```
1
2
3
```

⚠️ Ambiguous Topic: Iterable vs Iterator

Iterable	Iterator
Has `__iter__()`	Has `__next__()`
Can be looped	Produces next value
Example: list	Example: result of iter(list)

Interview Question:

Q: Why list is iterable but not an iterator?

Answer:

Because list has `__iter__()` but not `__next__()`.

🔹 PART 2: GENERATORS (Memory Efficient Iterators)

1️⃣ What is a Generator?

A generator is:

- A special function
- Uses `yield`
- Automatically implements iterator protocol

Example 1: Simple Generator

Python

```python
def simple_gen():
    yield 1
    yield 2
    yield 3

g = simple_gen()

print(next(g))
print(next(g))
print(next(g))
```

🔥 What Happens Internally?

When `yield` runs:

- Function pauses
- State is saved
- Resumes from same point

Example 2: Generator vs List Memory

Python

```python
def square_list(n):
    return [i*i for i in range(n)]

def square_gen(n):
```

```
    for i in range(n):
        yield i*i
```

Generator does NOT store all values in memory.

⚠️ Ambiguous Topic: yield vs return

return	yield
Ends function	Pauses function
Returns one value	Returns sequence lazily

Interview Question:

Q: Why are generators memory efficient?

Because they generate values on demand instead of storing entire sequence.

🔹 PART 3: CLOSURES (Advanced Concept)

What is a Closure?

A closure:

- Is a function inside another function
- Remembers outer variable even after outer function ends

Example 1: Basic Closure

Python

```python
def outer(x):
    def inner(y):
        return x + y
    return inner

add10 = outer(10)
```

```
print(add10(5))  # 15
```

🧠 Important Concept:

Even though `outer()` finished, `inner()` still remembers `x`.

How to Check Closure?

```python
print(add10.__closure__)
```

⚠️ Ambiguous Topic: Why use closures?

Used for:

- Data hiding
- Factory functions
- Decorators

Interview Question:

Q: How is closure different from global variable?

Closure stores state safely without polluting global namespace.

🔹 PART 4: DECORATORS (Powerful Concept)

What is a Decorator?

A decorator:

- Is a function
- Takes another function as argument
- Extends behavior without modifying original function

Example 1: Basic Decorator

Python

```python
def my_decorator(func):
    def wrapper():
        print("Before function call")
        func()
        print("After function call")
    return wrapper

@my_decorator
def say_hello():
    print("Hello")

say_hello()
```

Output:

Code

```
Before function call
Hello
After function call
```

What Does @ Do?

Python

```python
@my_decorator
def say_hello():
```

Is same as:

Python

```python
say_hello = my_decorator(say_hello)
```

Example 2: Decorator with Arguments

Python

```python
def repeat(n):
    def decorator(func):
        def wrapper():
            for _ in range(n):
                func()
        return wrapper
    return decorator

@repeat(3)
def greet():
    print("Hi")

greet()
```

⚠️ Ambiguous Topic: Why Use functools.wraps?

Without it:

- Function metadata lost
- **name** becomes wrapper

Fix:

```python
from functools import wraps

def my_decorator(func):
    @wraps(func)
    def wrapper():
        return func()
    return wrapper
```

🔥 Advanced Interview Questions

Q1: Difference between Generator and Iterator?

Generator:

- Created using function + yield
- Automatically iterator

Iterator:

- Class implementing **iter** and **next**

Q2: Can a generator be restarted?

No. Once exhausted, it cannot restart.

Q3: What is generator expression?

```python
g = (x*x for x in range(5))
```

Similar to list comprehension but lazy.

Q4: What is nonlocal keyword in closure?

Used to modify outer function variable.

Example:

```python
def counter():
    count = 0
    def inner():
        nonlocal count
        count += 1
        return count
    return inner
```

🧠 Concept Comparison Summary

Feature	Iterator	Generator	Closure	Decorator
Purpose	Iteration	Lazy iteration	Remember state	Modify behavior
Keyword	**next**	yield	nonlocal	@
Memory	Normal	Efficient	Small	Depends

Now let's make it interactive 👇

Question 1:

If I write:

```python
def test():
    yield 1
    return 5
```

What happens when:

- next() is called first time?
- next() is called second time?

Think and answer.

We'll go deeper into edge cases after your answer.

yes

Good 👍 Let's slow down and think properly.

We had:

```python
def test():
    yield 1
    return 5
```

Now imagine:

```python
g = test()
```

Step 1:

What happens when we call:

```python
next(g)
```

Don’t say “yes” 😄 — tell me:

👉 What value will it return?
👉 Does the function end here?

Answer those two first.

you do it

Nope 😄 I won’t do it for you — you’re in **study mode**, remember?

Let’s think it through together.

We have:

```python
def test():
    yield 1
    return 5
```

When we create:

```python
g = test()
```

Nothing runs yet. Correct?
Generators execute only when `next()` is called.

Now focus carefully:

When we call:

```python
next(g)
```

The function starts executing from top.

👉 It hits `yield 1`.

Now think:

- Does `yield` stop the function completely?
- Or does it pause and remember its state?

Answer this first:

Does the function finish after first `next(g)` or just pause?